

Programming Assignment: Multi-Client File Transfer

Assignment Weight: 5 marks

Duration: 2 Weeks

Background

Modern network applications such as WhatsApp, Telegram, FTP, and cloud storage systems use application-layer protocols built on top of transport-layer services.

In this assignment, you will design and implement a custom application-layer protocol that supports:

1. User authentication
 1. You will start with unencrypted username and password and we will enhance it in the next assignment
2. Multi-client communication
 1. The server should be multi-threaded and able to support multiple simultaneous clients
3. File transfer
 1. A client should be able to store a file on server or retrieve a file from server.
 2. A client should be able to send a file to another client using the server
4. Concurrent connections
 1. The need to be supported throughout.
 2. Your demo will require at least 5 concurrent clients.

Learning Outcomes

Upon successful completion, students will be able to:

- Design an application-layer protocol. (1 mark)

- Implement TCP socket communication with multiple clients concurrently. (1 mark)
- Transfer files over network sockets. (1 mark)
- Manage application-level messages and commands. (1 mark)
- Analyze performance and network behavior. (1 mark)

You have to design your own application protocol, remember that protocol is just a set of rules that define what is message syntax, semantics, rules to handle, etc.

Here is an example but you have to create your own.

=====

Messaging

Support:

MSG <username> <message>

Example:

MSG Alice Hello

The server forwards the message to Alice.

Broadcast Messaging

Support:

BROADCAST <message>

Example:

BROADCAST Exam postponed to Friday

All connected users receive the message.

File Transfer

Support:

UPLOAD <filename>

and

DOWNLOAD <filename>

Files must be stored in a server directory.

SEND <USER> <filename>

Files must be stored in a server directory.

User Listing

Support:

LIST

Return:

Alice

Bob

Charlie

Logout

Support:

LOGOUT

Client disconnects gracefully.

Protocol Design Requirement

Students must define a protocol specification document.

Example:

Command	Purpose
LOGIN username	Authenticate
MSG user text	Send private message
BROADCAST text	Send to all users

Server Responsibilities

The server must:

User Management

- Allow user login.
- Maintain a list of active users.
- Reject duplicate usernames.
- Do the operations asked by users, e.g. requesting or serving a file.

Client Responsibilities

The client application should:

- Connect using IP and Port.
- Authenticate with username.
- Send commands.
- Display incoming messages.
- Upload files.
- Download files.
- Send files
- Display server responses.

Technical Requirements

Programming Language: Python

Networking Requirements: TCP Sockets

Concurrency: Implement threading

Performance Experiment

Conduct experiments with:

Test 1

1 Client - send a message to server and receive a response

Measure:

- Response time (RTT)

Test 2

Send message from server to 5 Clients

Measure:

- Average response time

Test 3

10 Clients - send a file from server to clients. Change the size of the file to experiment (1 MB, 100 MB, 1 GB)

Measure:

- Server throughput

Deliverables

1. Source Code

Include:

Server Source Code

Client Source Code

2. Protocol Specification (2-3 Pages)

Describe:

- Message format
- Commands
- Error messages
- Sequence diagrams

3. Technical Report (5-8 Pages)

Include:

Introduction: Overview of application.

Design: Architecture diagrams.

Implementation: Socket programming techniques used.

Experimental Results: Performance measurements.

Prompts of AI use

Challenges: Problems faced and solutions.

Conclusion: Lessons learned.

AI Policy:

You can generate pieces of code but not the entire code with a single prompt. Your prompts must be submitted in your technical report.

You will also own all the code generated by you, i.e., you should be able to explain everything.